



CANARA
ENGINEERING COLLEGE

SUDHINDRA NAGARA, BENJANAPADAVU, BANTWAL TQ, D.K. - 574219



Department of Computer Science and Design



**Visual Application Development
and
Workflow Automation
Sub Code: 1BCDAL372**

Academic Year:2026-27

Sl.No	Experiments
1	Environment Setup & Relational Database Design - Set up the full lab environment, understand the LCNC ecosystem, and design a multi-table relational database for a College Event Management system using NocoDB's visual interface without writing SQL (Tools: Docker, NocoDB)
2	Form Design with Conditional Logic & Webhook Integration - Design a multi-page student feedback form with branching logic, connect it to an automation workflow via webhook, persist form responses to NocoDB, and create a branded thank-you landing page deployed publicly on GitHub Pages. (Tools: Formbricks, n8n, GrapesJS, GitHub Pages)
3	Full CRUD Application with Role-Based UI - Build a multi-screen Leave Management System in Appsmith that supports two roles (Student and Admin) and performs all four CRUD operations against the NocoDB backend using REST API calls. (Tools: Appsmith, NocoDB)
4	Multi-Step Workflow Automation with Error Handling - Build a Student Onboarding Automation workflow in n8n that triggers on new NocoDB records, validates data, sends personalized emails, handles errors gracefully, writes structured logs, and runs a scheduled daily summary — all without writing a backend service. (Tools: n8n, NocoDB, SMTP)
5	IoT Data Pipeline & Live Dashboard - Model a realistic multi-sensor IoT system using Node-RED flow-based programming. Simulate three sensors, classify readings into alert levels, suppress duplicate alerts, log events to file, and display a live dashboard with gauges, charts, and status indicators. (Tools: Node-RED, Node-RED Dashboard)
6	Zero-Code Analytics Dashboard with Data Storytelling - Build a comprehensive business intelligence dashboard in Metabase over a 500-row e-commerce dataset. Create 7 different question types, assemble a filterable dashboard, add narrative text cards, and embed one chart externally. (Tools: Metabase, PostgreSQL)
7	Responsive Website with Component System - Build a 4-page responsive product website using GrapesJS. Establish a reusable component library, define a consistent design token system, create a CSS animation block, and deploy all pages to GitHub Pages with functional inter-page navigation. (Tools: GrapesJS, GitHub Pages)
8	AI Chatbot with Document Q&A and Memory - Build a Retrieval Augmented Generation (RAG) chatbot in Flowise that answers questions from a college PDF document, maintains conversation memory across turns, runs entirely offline using Ollama, and is embedded into the Appsmith Leave Management app from Lab 3. (Tools: Flowise, Ollama, Llama 3)
9	Integrated Multi-Tool Application: Campus Event Portal - Build a complete Campus Event Portal that integrates four LCNC tools into one coherent system: NocoDB for data, Formbricks for public registration and feedback forms, n8n for event-driven automation (confirmation, reminders, feedback triggers), and Appsmith for the admin dashboard. No data should be entered twice — all four tools must communicate with each other. (Tools: Appsmith, NocoDB, n8n, Formbricks)
10	Product Sprint: Design, Build & Demo a Complete LCNC Product - Ship a complete, working LCNC product in 3 hours under timed milestone constraints. Each group builds a different assigned problem from scratch, demonstrating their ability to decompose a real-world problem into data, UI, automation, and analytics layers using no-code tools. (all tools)

Visual Application Development and Workflow Automation

LABORATORY MANUAL

3rd Semester — Computer Science & Design

Scheme 2025

A hands-on laboratory course where students design and deploy complete software systems — including databases, user interfaces, automation pipelines, analytics dashboards, and AI assistants — using industry-standard visual development and workflow automation platforms, without writing traditional application code.

10 Laboratory Experiments

NocoDB · Appsmith · n8n · Formbricks · Node-RED · Metabase · GrapesJS · Flowise

Fully Free & Open Source Stack

Course Outcomes & Program Outcomes Mapped

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	3	2										3	
CO2			3		3							2	3
CO3			3	2								3	1
CO4		3								2		2	2
CO5						2			2		3	3	2

CO1: Design and implement relational data models and REST API-backed applications using visual database and application builder platforms.

Mapped POs:

- **PO1 — Engineering Knowledge:** Students apply principles of relational database design (normalization, foreign keys, constraints) and REST API architecture to build functional data-driven applications.
- **PO2 — Problem Analysis:** Students analyze a given problem domain, decompose it into entities and relationships, and select appropriate field types, constraints, and views to model it correctly in NocoDB.
- **PSO1 — Full Stack Applications:** Students apply core database design principles — entity identification, normalization, referential integrity, and constraint definition — to build a structured data layer and expose it as a REST API. This directly maps to the data and backend layers of full-stack application development.

Labs: Lab 1 (NocoDB schema design), Lab 3 (Appsmith CRUD application)

CO2: Build multi-screen, role-based user interfaces by integrating visual frontend builders with backend data sources through API connections.

Mapped POs:

- **PO3 — Design/Development of Solutions:** Students design and develop complete multi-screen applications with role-based access control, form validation, and dynamic data binding — demonstrating solution design across the full application stack.
- **PO5 — Modern Tool Usage:** Students demonstrate proficiency in using contemporary LCNC platforms (Appsmith, GrapesJS, Formbricks) as tools to design, prototype, and deploy software solutions efficiently.
- **PSO1 — Full Stack Applications:** Students connect frontend UI components to backend REST APIs using HTTP methods (GET, POST, PATCH, DELETE), implement client-server data binding, and manage application state across screens — demonstrating full-stack integration principles.
- **PSO2 — Multimedia, Graphics and User Interface:** Students design role-based multi-screen interfaces in Appsmith, build responsive multi-page websites in GrapesJS using design token systems and component libraries, and create conditional forms in Formbricks — directly applying User Interface design principles including layout, hierarchy, responsiveness, and interaction design.

Labs: Lab 2 (Formbricks + GrapesJS), Lab 3 (Appsmith multi-role UI), Lab 7 (GrapesJS responsive website)

CO3: Design and deploy event-driven automation workflows incorporating conditional logic, error handling, scheduled triggers, and multi-system integration.

Mapped POs:

- **PO3 — Design/Development of Solutions:** Students design multi-step automation pipelines that mirror real-world business process automation, applying systematic engineering thinking to workflow decomposition and exception handling.

- **PO4 — Conduct Investigations:** Students test automation workflows by deliberately introducing invalid data and edge cases, observe system behavior, analyze failure modes, and iteratively refine the workflow — mirroring an investigative engineering approach.
- **PSO1 — Full Stack Applications:** Students design backend integration pipelines that connect multiple systems through APIs and webhooks — modeling the server-side and integration layers of full-stack architecture. Workflow design in n8n and Node-RED requires understanding of event-driven systems, data transformation, and exception handling, all of which are foundational full-stack engineering concepts.

Labs: Lab 4 (n8n multi-step automation), Lab 5 (Node-RED IoT pipeline), Lab 9 (integrated event-driven portal)

CO4: Construct data analytics dashboards by querying structured datasets, applying filters, and communicating data-driven insights through appropriate visualizations.

Mapped POs:

- **PO2 — Problem Analysis:** Students analyze a business dataset, identify meaningful patterns and KPIs, select appropriate chart types for each metric, and structure a coherent narrative — demonstrating analytical reasoning applied to real data.
- **PO10 — Communication:** Students produce dashboards with text narrative cards that communicate analytical findings clearly to non-technical stakeholders, demonstrating the ability to translate technical output into accessible business insight.
- **PSO1 — Full Stack Applications:** Students query a PostgreSQL backend through Metabase's visual query builder, applying grouping, aggregation, sorting, and filtering operations that mirror SQL-level data manipulation — engaging with the data layer of full-stack systems analytically.
- **PSO2 — Multimedia, Graphics and User Interface:** Students select and configure chart types (line, bar, pie, area, combo), arrange visual dashboard layouts, apply interactive filters, and write narrative text cards — directly applying data visualization and information design principles within the Multimedia and User Interface scope of PSO2.

Labs: Lab 6 (Metabase BI dashboard with data storytelling)

CO5: Integrate multiple LCNC platforms into a cohesive software product by orchestrating data flow, automation, and user interfaces across tool boundaries, and demonstrate the product through a structured sprint.

Mapped POs:

- **PO6 — The Engineer and Society:** Students build systems that address real institutional needs — hostel complaint tracking, placement portals, equipment booking — demonstrating awareness of how software solutions serve societal and organizational contexts.
- **PO9 — Individual and Team Work:** Lab 9 and Lab 10 require structured group collaboration with defined ownership of each tool layer, scheduled milestone reviews, and a live demo — developing both individual technical accountability and team coordination skills.
- **PO11 — Project Management and Finance:** Lab 10 is a timed product sprint with defined milestones, deliverable checklists, and an evaluation rubric — directly developing the ability to plan, execute, and deliver a software product within time and scope constraints.
- **PSO1 — Full Stack Applications:** The product sprint requires students to architect and deliver a complete system spanning all layers — relational data (NocoDB), user interface (Appsmith), form layer (Formbricks), automation (n8n), and analytics (Metabase). This is the most comprehensive demonstration of full-stack application thinking in the course, with students making architectural decisions about how data flows across all layers.
- **PSO2 — Multimedia, Graphics and User Interface:** Every sprint deliverable includes user-facing interface artifacts — an Appsmith admin dashboard, a Formbricks public form, and a Metabase analytics view — all of which require deliberate User Interface design decisions about layout, usability, and information presentation.

Labs: Lab 8 (Flowise RAG chatbot), Lab 9 (4-tool integrated portal), Lab 10 (product sprint)

Overview

This laboratory manual is designed for 3rd Semester Computer Science and Design Engineering students. The lab introduces Low Code / No Code (LCNC) platforms — a paradigm where complete, production-ready software systems are assembled through visual tools, drag-and-drop interfaces, and configuration rather than hand-written code.

All tools in this lab are 100% free and open-source. The entire stack runs locally via Docker Compose, requiring no paid API keys, no cloud subscriptions, and no internet connection after initial setup.

Technology Stack Quick Reference

Tool	Purpose & Access URL
NocoDB	Visual relational database builder → http://localhost:8080
Appsmith	Drag-and-drop internal app builder → http://localhost:8081
n8n	Visual workflow automation engine → http://localhost:5678
Formbricks	Advanced form builder with logic → http://localhost:3000
Node-RED	Flow-based IoT/data programming → http://localhost:1880
Metabase	Zero-code BI & dashboards → http://localhost:3001
GrapesJS	Visual HTML/CSS website builder → Browser-based (no install)
Flowise	Visual LLM / AI chatbot builder → http://localhost:3002
Ollama	Local LLM runtime (Llama 3) → http://localhost:11434
PostgreSQL	Shared database backend → localhost:5432

Lab Schedule

Lab No.	Title
Lab 1	Environment Setup + Relational Database Design (NocoDB)
Lab 2	Form Design with Conditional Logic & Webhook Integration
Lab 3	Full CRUD Application with Role-Based UI (Appsmith)
Lab 4	Multi-Step Workflow Automation with Error Handling (n8n)
Lab 5	IoT Data Pipeline & Live Dashboard (Node-RED)
Lab 6	Zero-Code Analytics Dashboard with Data Storytelling (Metabase)
Lab 7	Responsive Website with Component System (GrapesJS)
Lab 8	AI Chatbot with Document Q&A and Memory (Flowise + Ollama)
Lab 9	Integrated Multi-Tool Application: Campus Event Portal
Lab 10	Product Sprint: Design, Build & Demo a Complete LCNC Product

Setup Files

File	Purpose & When to Use
lcnc-lab-setup.bat	ONE-TIME SETUP — Run this once on each new machine. Downloads Docker & Git if needed, extracts all config files, pulls Docker images, starts the stack, loads sample data, downloads AI models.
start-lcnc-lab.bat	START SESSION — Run at the beginning of every lab session. Starts Docker Desktop if needed, removes stale containers, starts all services.
stop-lcnc-lab.bat	STOP SESSION — Run at the end of every lab session. Gracefully stops all containers. All student data is preserved.
fresh-start.bat	EMERGENCY RESET — Use only if the stack is broken and won't start. Wipes all containers and volumes, rebuilds from scratch. WARNING: deletes all data.

Database Credentials

Parameter	Value
Host (from host machine)	localhost
Host (from inside Docker)	postgres
Port	5432
Admin user	lcncadmin
Password	lcncpass123
NocoDB database	nocodb
n8n database	n8ndb
Formbricks database	formbricksdb
Metabase database	metabasedb
Flowise database	flowisedb

SECTION 1 System Requirements

Hardware Requirements

Component	Requirement
Operating System	Windows 11 (Home, Pro, or Education)
RAM	Minimum 8 GB — 16 GB recommended for running all services simultaneously
Storage	Minimum 25 GB free disk space (Docker images + volumes)
CPU	Minimum 4 cores with hardware virtualization support (Intel VT-x or AMD-V)
Internet	Required during first-time setup only — approximately 13-15 GB total download

Verifying Your System

Check RAM

- Step 1:** Press Ctrl + Shift + Esc to open Task Manager.
- Step 2:** Click the Performance tab → Memory.
- Step 3:** Confirm Total shows at least 8.0 GB.

Check Free Disk Space

- Step 1:** Open File Explorer → This PC.
- Step 2:** Right-click Local Disk (C:) → Properties.
- Step 3:** Confirm Free Space shows at least 25 GB.
- Step 4:** If the lab folder will be on a different drive (e.g. E:), check that drive instead.

Verify Virtualization is Enabled

- Step 1:** Press Ctrl + Shift + Esc → Performance tab → CPU.
- Step 2:** Look at the bottom right of the CPU panel.
- Step 3:** Confirm Virtualization: Enabled.
- Step 4:** If it shows Disabled: restart the PC, enter BIOS (F2 / F10 / Del during startup), find Intel VT-x or AMD-V / SVM Mode, enable it, save and exit.

SECTION 2 First-Time Setup

This section covers the complete setup process for a new machine. The entire process is automated through a single batch file.

What the Setup Script Does

Step	Action
1	Checks system requirements — RAM, disk space, Windows version
2	Installs Docker Desktop if not already installed
3	Installs Git for Windows if not already installed
4	Asks for the lab folder path, creates it, extracts all config files
5	Pulls all Docker images (~8-10 GB — the longest step)
6	Starts PostgreSQL and waits for it to be fully healthy
7	Starts all remaining lab services
8	Loads sample e-commerce data for the Metabase lab
9	Downloads the Llama 3 AI model (~4.7 GB) for the Flowise lab
10	Installs the Node-RED Dashboard plugin
11	Creates Start, Stop, and Fresh Start shortcuts on the Desktop

Running the Setup

Step 1: Download lcnc-lab-setup.bat to any location on the machine (Desktop is fine).

Step 2: Right-click lcnc-lab-setup.bat and select Run as administrator.

Step 3: Click Yes on the UAC (User Account Control) prompt.

Step 4: When prompted for the lab folder path, either:

- Press Enter to use the default: C:\Users\YourName\Documents\lcnc-lab
- Type a custom path and press Enter, e.g.: E:\lcnc-lab

Step 5: The script will run all steps automatically. Do not close the window.

Step 6: When Docker Desktop is being installed, a UAC prompt will appear — click Yes.

Step 7: Total time: 30-60 minutes depending on internet speed.

Step 8: When complete, three Desktop shortcuts are created: Start LCNC Lab, Stop LCNC Lab, Fresh Start LCNC Lab.

NOTE: If Docker Desktop or Git are already installed, the script detects them automatically and skips those installation steps.

TIP: To speed up setup on multiple machines: after the first machine completes setup, copy DockerDesktopInstaller.exe and GitInstaller.exe from the lab folder alongside lcnc-lab-setup.bat. Subsequent machines will use these cached files instead of downloading.

Offline Installation (No Internet)

For college labs with restricted internet access, pre-download everything on one machine and distribute via USB:

Step 1: Run the setup on one internet-connected machine. Let it complete fully.

Step 2: Copy the entire lab folder (e.g. E:\lcnc-lab) to a USB drive. It contains all extracted files and cached installers.

Step 3: Also copy the Docker images using: `docker save $(docker images -q) -o lcnc-images.tar`

Step 4: On each offline machine: copy the lab folder from USB, load Docker images: `docker load -i lcnc-images.tar`

Step 5: Run `lcnc-lab-setup.bat` — it will find all cached installers and skip all downloads.

NOTE: The Docker images tar file will be approximately 12-15 GB. Use a USB 3.0 drive for faster transfer.

SECTION 3 First Visit — Account Creation

After setup completes, open each tool in the browser and create accounts. This is a one-time process — accounts persist across sessions in Docker volumes.

NocoDB — <http://localhost:8080>

- Step 1:** Click Sign Up.
- Step 2:** Enter any email (e.g. admin@lcnclab.local) and a password.
- Step 3:** Click Sign Up. You land on the project dashboard.

Appsmith — <http://localhost:8081>

- Step 1:** Fill in your name, email, and password on the welcome screen.
- Step 2:** Click Get Started.
- Step 3:** Skip the onboarding tour if prompted.

n8n — <http://localhost:5678>

Pre-configured. Login directly with: Username: admin | Password: admin123

Formbricks — <http://localhost:3000>

- Step 1:** Click Create account.
- Step 2:** Enter your name, email, and password.
- Step 3:** When prompted for a team name, enter: LCNC Lab
- Step 4:** Skip the onboarding steps.

Node-RED — <http://localhost:1880>

No login required. Opens directly to the flow editor.

Install the Dashboard Plugin (required for Lab 5)

- Step 1:** Click the hamburger menu (≡) at the top right.
- Step 2:** Select Manage palette.
- Step 3:** Click the Install tab.
- Step 4:** Search: node-red-dashboard
- Step 5:** Click Install next to node-red-dashboard by Dave Conway-Jones.
- Step 6:** Wait 1-2 minutes. New dashboard nodes appear in the left panel.
- Step 7:** The live dashboard will be accessible at: <http://localhost:1880/ui>

Metabase — <http://localhost:3001>

- Step 1:** Click Let's get started on the welcome screen.
- Step 2:** Enter your name, email, and password.
- Step 3:** On the Add your data step, select PostgreSQL and enter:
 - Display name: LCNC Lab Database

- Host: postgres
- Port: 5432
- Database name: nocodb
- Username: lcncadmin
- Password: lcncpass123

Step 4: Click Next → Finish.

NOTE: Metabase is a Java application and takes 3-5 minutes to fully start on first boot. If the page shows a loading screen, wait and refresh.

Flowise — <http://localhost:3002>

Pre-configured. Login directly with: Username: admin | Password: admin123

Llama 3 AI Model (for Lab 8)

If the model was not downloaded during setup, pull it manually:

```
docker exec -it lcnc_ollama ollama pull llama3
docker exec -it lcnc_ollama ollama pull nomic-embed-text
```

Verify both are downloaded:

```
docker exec -it lcnc_ollama ollama list
```

NOTE: The llama3 model is approximately 4.7 GB and nomic-embed-text is approximately 270 MB. Both run entirely offline once downloaded.

SECTION 4 Daily Session Management

Starting a Session

- Step 1:** Double-click the Start LCNC Lab shortcut on the Desktop.
- Step 2:** Click Yes on the UAC prompt.
- Step 3:** When prompted for the lab folder path, press Enter to use the default or type the custom path.
- Step 4:** The script starts Docker Desktop automatically if it is not running.
- Step 5:** Wait for all containers to show as running (approximately 60-90 seconds).
- Step 6:** Open your browser and navigate to any tool URL.

TIP: All student work from the previous session is automatically available — tables, workflows, apps, and dashboards are all preserved in Docker volumes.

Stopping a Session

- Step 1:** Double-click the Stop LCNC Lab shortcut on the Desktop.
- Step 2:** When prompted, press Enter to use the default path or type the custom path.
- Step 3:** All containers stop gracefully. Student data is fully preserved.

WARNING: Never run `docker compose down -v` at the end of a session. This deletes all volumes and all student work. Use Stop LCNC Lab shortcut instead.

Useful Docker Commands

Open a terminal (Command Prompt or PowerShell) from the lab folder to run these:

Command	What it does
<code>docker compose ps</code>	Show status of all containers
<code>docker compose logs -f nocodb</code>	Watch live logs for a specific service (replace nocodb with any service name)
<code>docker compose restart nocodb</code>	Restart a single service
<code>docker compose stop</code>	Stop all services (data preserved)
<code>docker compose start</code>	Start all previously stopped services
<code>docker compose down --remove-orphans</code>	Remove stale containers (data preserved)
<code>docker system prune</code>	Clean up unused images to free disk space (safe — does not touch volumes)
<code>docker compose down -v</code>	DANGER: Wipes all data permanently

SECTION 5 Multi-Machine Lab Deployment

Strategy A — Internet Available (Simplest)

Each machine downloads everything independently. Best when all lab machines have fast internet access.

Step 1: Copy `lcnc-lab-setup.bat` to each machine (USB drive, network share, or email).

Step 2: Right-click → Run as administrator on each machine.

Step 3: Each machine downloads Docker, Git, and all Docker images independently.

Step 4: Time per machine: 30-60 minutes.

TIP: Run setup on all machines simultaneously to save time — each machine downloads in parallel.

Strategy B — USB Distribution (No Internet Required)

Pre-download everything once and distribute via USB. Best for labs with no or slow internet.

Step 1: Run the full setup on one internet-connected machine.

Step 2: After setup completes, copy `DockerDesktopInstaller.exe` and `GitInstaller.exe` from the lab folder to a USB drive alongside `lcnc-lab-setup.bat`.

Step 3: Save Docker images to a file:

```
docker save -o lcnc-docker-images.tar nocodb/nocodb appsmith/appsmith-ce
n8nio/n8n ghcr.io/formbricks/formbricks nodered/node-red metabase/metabase
flowiseai/flowise ollama/ollama axllent/mailpit postgres:15-alpine
```

Step 4: Copy `lcnc-docker-images.tar` to the USB drive (~12-15 GB).

Step 5: On each target machine:

- Install Docker Desktop using `DockerDesktopInstaller.exe` from the USB
- Load Docker images: `docker load -i lcnc-docker-images.tar`
- Place `lcnc-lab-setup.bat` alongside the installer files and run it — it detects everything cached

NOTE: A USB 3.0 drive is strongly recommended. Loading the 12-15 GB tar file takes approximately 5-10 minutes per machine.

Strategy C — Network Share

Place `lcnc-lab-setup.bat` and cached installers on a shared network drive. Each machine runs the script from the share.

Step 1: Copy the setup script and installers to `\\server\lcnc-lab\`.

Step 2: Each student machine runs: `\\server\lcnc-lab\lcnc-lab-setup.bat` as administrator.

Step 3: The script prompts for the local lab folder path — students enter a local path like `E:\lcnc-lab`.

Step 4: Config files are extracted locally. Docker images still need internet or pre-loaded tar.

SECTION 6 Troubleshooting

Container Conflicts

Error: The container name /lcnc_nodered is already in use. This happens when containers from a previous session are still registered. Fix:

```
docker rm -f lcnc_postgres lcnc_nocodb lcnc_appsmith lcnc_n8n lcnc_formbricks
lcnc_nodered lcnc_metabase lcnc_flowise lcnc_ollama lcnc_mailpit
docker compose up -d
```

Service Not Loading in Browser

If a URL gives a connection refused or timeout error after startup:

Step 1: Check container status:

```
docker compose ps
```

Step 2: If the container shows restarting or exited, check its logs:

```
docker compose logs --tail=30 <service-name>
```

Step 3: Wait — Metabase takes 3-5 minutes to start. Appsmith takes 2-3 minutes.

Step 4: Restart the specific service:

```
docker compose restart <service-name>
```

PostgreSQL Not Healthy

Error: dependency failed to start: container lcnc_postgres is unhealthy

Step 1: Check postgres logs:

```
docker logs lcnc_postgres --tail=20
```

Step 2: If you see uuid_generate_v4 does not exist, apply the fix:

```
docker exec -it lcnc_postgres psql -U lcncadmin -d postgres -c "CREATE
EXTENSION IF NOT EXISTS 'uuid-osspl';"
```

Step 3: If the database is corrupted from a previous broken run, wipe and restart:

```
docker compose down -v
docker compose up -d
```

Missing Databases

If a service cannot connect because its database does not exist (common when postgres volume existed before the new config):

```
docker exec -it lcnc_postgres psql -U lcncadmin -d postgres -c "CREATE DATABASE
n8ndb;"
docker exec -it lcnc_postgres psql -U lcncadmin -d postgres -c "CREATE DATABASE
formbricksdb;"
docker exec -it lcnc_postgres psql -U lcncadmin -d postgres -c "CREATE DATABASE
metabasedb;"
docker exec -it lcnc_postgres psql -U lcncadmin -d postgres -c "CREATE DATABASE
flowisedb;"
docker exec -it lcnc_postgres psql -U lcncadmin -d postgres -c "CREATE DATABASE
nocodb;"
```

Then restart the affected services:

```
docker compose restart nocodb n8n formbricks metabase flowise
```

Docker Desktop Not Starting

If Docker Desktop fails to start or shows an error:

Step 1: Verify WSL 2 is installed and set as default:

```
wsl --status
```

Step 2: If WSL kernel is outdated:

```
wsl --update
```

Step 3: Restart Docker Desktop from the Start Menu manually.

Step 4: If still failing, restart the machine and try again.

Port Already in Use

Error: Bind for 0.0.0.0:8080 failed: port is already allocated

Step 1: Find what is using the port (replace 8080 with the conflicting port):

```
netstat -ano | findstr :8080
```

Step 2: Note the PID (last column). Open Task Manager → Details tab → find the PID → End Task.

Step 3: Run docker compose up -d again.

Full System Reset

If the entire stack is broken and nothing else works:

Step 1: Double-click the Fresh Start LCNC Lab shortcut on the Desktop.

Step 2: Type the lab folder path when prompted and press Enter.

Step 3: The script wipes all containers and volumes, then rebuilds from scratch.

WARNING: Fresh Start deletes ALL student data permanently. Only use it as a last resort or at the start of a new semester.

Common Issues Quick Reference

Problem	Fix
URL shows connection refused	Wait 60-90 seconds after start — services need time to initialize
Metabase shows loading screen	Wait 3-5 minutes — it is a Java app with longer startup time
Container conflict on startup	Run: <code>docker rm -f lcnc_postgres lcnc_nocodb ...</code> then <code>docker compose up -d</code>
uuid_generate_v4 error	Run the extension fix command on the postgres container
Formbricks migration error	Delete the failed migration from the database — see Section 6
Ollama model not found	Run: <code>docker exec -it lcnc_ollama ollama pull llama3</code>
docker command not found	Docker Desktop is not running — start it from the system tray
Low disk space warning	Run: <code>docker system prune</code> to remove unused images
WSL error on Windows	Run: <code>wsl --update</code> in PowerShell as administrator

SECTION 7 Semester & Maintenance

End of Semester Reset

At the end of each semester, wipe all student data and reset the stack for the next batch:

Step 1: Optionally backup any student work you want to keep (export from NocoDB, Appsmith, etc.).

Step 2: Run the Fresh Start shortcut on each lab machine.

Step 3: All volumes are wiped. The stack rebuilds with clean databases.

Step 4: Students re-create accounts on first visit of the new semester.

Updating Docker Images

Periodically update all images to get bug fixes and security patches:

```
cd E:\lcnc-lab
docker compose pull
docker compose up -d
```

NOTE: Pulling updates requires internet. Do this during a maintenance window, not during a live lab session.

Backup Student Work

If you need to preserve student work before a reset:

NocoDB — Export Tables

Step 1: Open NocoDB → select the table.

Step 2: Click the download icon → Export as CSV or Excel.

n8n — Export Workflows

Step 1: Open n8n → select the workflow.

Step 2: Click the three-dot menu → Download.

Step 3: Saves as a .json file that can be reimported.

Full Database Backup

```
docker exec lcnc_postgres pg_dump -U lcncadmin nocodb > nocodb_backup.sql
docker exec lcnc_postgres pg_dump -U lcncadmin n8ndb > n8n_backup.sql
```

File Reference

File	Location & Description
lcnc-lab-setup.bat	Any location — the all-in-one first-time setup script
start-lcnc-lab.bat	Lab folder + Desktop shortcut — run at start of every session
stop-lcnc-lab.bat	Lab folder + Desktop shortcut — run at end of every session
fresh-start.bat	Lab folder + Desktop shortcut — emergency full reset
docker-compose.yml	Lab folder — defines all services. Do not edit unless instructed.
init-db.sql	Lab folder — creates databases on first postgres start. Do not edit.
LCNC_Lab_Manual.docx	Distribute to students — contains all 10 lab experiments

Section 8 Global Environment Setup

This section covers the one-time setup that must be completed before Lab 1. Each student workstation (or shared lab server) must complete these steps exactly once.

System Requirements

OS	Ubuntu 22.04 / Debian 12 / Windows 11 with WSL2 / macOS 13+
RAM	Minimum 8 GB (16 GB recommended for running all services simultaneously)
Storage	Minimum 20 GB free disk space for Docker images and volumes
CPU	4 cores minimum
Network	Internet access required only during initial docker pull

Step-by-Step: Install Docker & Docker Compose

On Ubuntu / Debian (run in terminal):

```
# Remove old Docker versions if any
sudo apt-get remove docker docker-engine docker.io containerd runc
```

```
# Install prerequisites
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg lsb-release
```

```
# Add Docker GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

```
# Add Docker repository
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
```

```
# Install Docker Engine and Compose plugin
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

```
# Add your user to docker group (avoid using sudo every time)
sudo usermod -aG docker $USER
newgrp docker
```

```
# Verify installation
docker --version
docker compose version
```

NOTE: On Windows 11: Install Docker Desktop from <https://www.docker.com/products/docker-desktop/> — it includes WSL2 and Docker Compose automatically.

Step-by-Step: Start the Lab Stack

Step 1: Download or clone the lab repository to your machine:

```
git clone https://github.com/your-institution/lcnc-lab.git
cd lcnc-lab
```

Step 2: Pull all Docker images (requires internet — do this once in lab before sessions):

```
docker compose pull
```

Step 3: Start all services:

```
docker compose up -d
```

Step 4: Verify all containers are running:

```
docker compose ps
```

You should see all 8 containers with status 'running' or 'healthy'.

Step 5: Pull the Llama 3 model into Ollama (one-time, ~4.7 GB):

```
docker exec -it lcnc_ollama ollama pull llama3
```

Step 6: Open your browser and verify each tool loads at its URL (see Quick Reference table above).

TIP: To stop all services at end of session: `docker compose stop`

TIP: To start again next session: `docker compose start`

TIP: All data persists in Docker volumes — student work is saved across sessions.

Quick Reference

Service URLs & Credentials

Service	URL	Login
NocoDB	http://localhost:8080	Create account on first visit
Appsmith	http://localhost:8081	Create account on first visit
n8n	http://localhost:5678	admin / admin123
Formbricks	http://localhost:3000	Create account on first visit
Node-RED	http://localhost:1880	No login required
Metabase	http://localhost:3001	Setup wizard on first visit
Flowise	http://localhost:3002	admin / admin123
Mailpit	http://localhost:8025	No login required

LAB 1 Environment Setup + Relational Database Design

Tools: Docker, NocoDB

Duration	3 hours
Tools	Docker, NocoDB (http://localhost:8080)
Prerequisite	Global environment setup completed
Difficulty	Medium

Objective

Set up the full lab environment, understand the LCNC ecosystem, and design a multi-table relational database for a College Event Management system using NocoDB's visual interface without writing SQL.

Theory

Traditional database design requires knowledge of SQL DDL (CREATE TABLE, ALTER TABLE, foreign key constraints). NocoDB provides a visual layer over standard relational databases (PostgreSQL in our case), letting you define tables, field types, constraints, and relationships through a spreadsheet-like interface while generating proper SQL underneath.

NocoDB exposes every table automatically as a REST API with full CRUD support, Swagger documentation, and support for filtering, sorting, pagination, and aggregation — making it suitable as a complete backend for small to medium applications.

Part A: NocoDB Initial Setup

Step 1: Open NocoDB at <http://localhost:8080>

Step 2: Click 'Sign Up' and create an account with your email and password.

Step 3: After login, click '+ New Project' → choose 'Create New Project'.

Step 4: Name the project 'Event Management' and click Create.

Step 5: You will see the empty project dashboard. This is your NocoDB workspace.

NOTE: NocoDB is connected to the PostgreSQL container running at `postgres:5432`. All data you create is stored persistently in the Docker volume.

Part B: Design the Database Schema

You will create 4 linked tables for a College Event Management system.

Table 1: Events

Step 1: Click '+ Add new table' and name it 'Events'.

Step 2: Add the following fields by clicking '+ Add column':

Field Name	Field Type & Configuration
Title	Single line text — Mark as Required
Description	Long text
Event Date	Date — Required
Start Time	Time
End Time	Time
Max Capacity	Number — Default: 100
Registration Open	Checkbox — Default: true
Event Type	Single Select — Options: Workshop, Seminar, Hackathon, Cultural, Sports
Banner URL	URL
Created At	Created time (auto)

Step 3: Click 'Save' after adding all fields.

Table 2: Venues

Step 1: Add a new table named 'Venues'. Add these fields:

Field Name	Field Type & Configuration
Venue Name	Single line text — Required
Building	Single line text
Floor	Number
Seating Capacity	Number — Required
Has Projector	Checkbox
Has AC	Checkbox
Contact Person	Single line text
Contact Phone	Phone number

Table 3: Participants

Step 1: Add a new table named 'Participants'. Add these fields:

Field Name	Field Type & Configuration
Full Name	Single line text — Required
USN	Single line text — Required, Unique
Email	Email — Required
Branch	Single Select — Options: CSE, ISE, ECE, ME, CV, DS
Semester	Number
Phone	Phone number
Registered At	Created time (auto)

Table 4: Registrations (Junction Table)

Step 1: Add a new table named 'Registrations'. Add these fields:

Field Name	Field Type & Configuration
Status	Single Select — Options: Confirmed, Waitlisted, Cancelled — Default: Confirmed
Payment Status	Single Select — Options: Paid, Free, Pending
Check-In	Checkbox — Default: false
Notes	Long text
Registered At	Created time (auto)

Part C: Create Relationships

Now link the tables together to model foreign key relationships visually.

Step 1: Open the 'Events' table. Click '+ Add column'.

Step 2: Choose field type 'Link to another Record'. Set linked table to 'Venues'. Name the field 'Venue'.

Step 3: This creates a one-to-many relationship: one Venue can host many Events.

Step 4: Open the 'Registrations' table. Add a 'Link to another Record' field linking to 'Events'. Name it 'Event'.

Step 5: Add another 'Link to another Record' field linking to 'Participants'. Name it 'Participant'.

Step 6: The Registrations table now acts as a junction table connecting Events and Participants.

TIP: After creating links, NocoDB automatically creates reverse lookup fields in the linked tables. You will see 'Registrations' appear as a field in both Events and Participants tables.

Part D: Populate with Sample Data

Step 1: Add at least 3 Venue records with realistic data.

Step 2: Add at least 5 Event records — link each to a venue by clicking the Venue cell and selecting from the list.

Step 3: Add at least 10 Participant records with your classmates' details.

Step 4: Add at least 8 Registration records — link each to an Event and a Participant.

Part E: Create Views

Views let you see the same data in different formats without duplicating records.

Step 1: In the Events table, click the '+ Add View' button on the left sidebar.

Step 2: Create a 'Gallery View' — you will see event cards. Set the Banner URL field as the image field.

Step 3: Create a 'Calendar View' — set 'Event Date' as the date field. Events appear on the calendar.

Step 4: Create a 'Filtered View' named 'Upcoming Events' — add a filter: Event Date is after today. Only future events appear.

Step 5: In the Registrations table, create a view filtered by Status = Confirmed.

Part F: Explore the REST API

Step 1: Click the '?' icon at the top right → 'Swagger'.

Step 2: Expand the Events section. Click 'GET /api/v1/db/data/noco/{projectId}/Events'.

Step 3: Click 'Try it out' → Execute. Observe the JSON response.

Step 4: Try a POST request to create a new Event through the Swagger UI.

Step 5: Copy the API token from Settings → API Tokens → Generate. You will need this in Lab 3.

TIP: Keep the API token saved in a text file — you will use it in Labs 3, 4, and 9.

OUTCOME: Students have set up the full LCNC lab stack and built a production-quality relational database schema with 4 tables, foreign key relationships, multiple views, and a working REST API — entirely without writing SQL.

Evaluation Rubric

Criterion	Marks	Description
Docker setup and all services running	10	All 8 containers show status healthy in docker compose ps

Criterion	Marks	Description
Correct schema with all required fields and types	20	4 tables created, field types appropriate, required fields marked
Relationships correctly established	20	All 3 links created and visible in both linked tables
Sample data populated (minimum counts met)	15	3 venues, 5 events, 10 participants, 8 registrations
All 4 views created and functional	20	Gallery, Calendar, Filtered, and Status-filtered views working
API exploration with successful GET and POST	15	Swagger screenshots showing successful API calls

LAB 2 Form Design with Conditional Logic & Webhook Integration

Tools: Formbricks · n8n · GrapesJS · GitHub Pages

Duration	3 hours
Tools	Formbricks (port 3000), n8n (port 5678), GrapesJS (browser), GitHub Pages
Prerequisite	Lab 1 completed; NocoDB API token available
Difficulty	Medium

Objective

Design a multi-page student feedback form with branching logic, connect it to an automation workflow via webhook, persist form responses to NocoDB, and create a branded thank-you landing page deployed publicly on GitHub Pages.

Theory

Modern form design goes far beyond simple input fields. Conditional logic (show/hide fields based on previous answers) creates adaptive forms that feel personalized and collect more relevant data. Webhooks enable real-time integration: when a user submits a form, an HTTP POST is immediately fired to another service, enabling zero-delay automation.

Part A: Formbricks Initial Setup

- Step 1:** Open Formbricks at <http://localhost:3000>
- Step 2:** Click 'Sign Up' and create an admin account.
- Step 3:** You will be prompted to create a Team. Name it 'LCNC Lab'.
- Step 4:** From the dashboard, click '+ New Survey'.
- Step 5:** Choose 'Start from scratch' and select 'Link Survey' type.
- Step 6:** Name the survey 'Student Feedback — Event Management Portal'.

Part B: Design the Multi-Page Feedback Form

Build a 4-page form with at least 12 questions and conditional branching.

Page 1: Identity

- Step 1:** Add a 'Welcome Card' block: Title: 'Event Feedback Form', Subtitle: 'Your feedback helps improve campus events. This takes 3 minutes.'

Step 2: Add a 'Short Text' question: 'Your Full Name' — Mark as Required.

Step 3: Add a 'Short Text' question: 'Your USN (e.g., 1XX21CS001)' — Required.

Step 4: Add a 'Single Select' question: 'Your Branch' — Options: CSE, ISE, ECE, ME, CV, DS.

Page 2: Event Rating

Step 1: Add a 'Rating' question (1-5 stars): 'How would you rate this event overall?'

Step 2: Add a 'Rating' question: 'Rate the venue and facilities.'

Step 3: Add a 'Rating' question: 'Rate the relevance of the content to your learning.'

Step 4: Add a 'Multi Select' question: 'What did you enjoy most?' — Options: Speaker Quality, Networking, Hands-on Activities, Free Food, Event Organization, Prize/Certificate.

Page 3: Conditional — Negative Feedback (shown only if overall rating is 1 or 2)

Step 1: Add a new page. Click the page settings → 'Add Logic'.

Step 2: Set condition: 'If Question: Overall Rating is less than 3, show this page. Otherwise skip to Page 4.'

Step 3: Add a 'Long Text' question: 'What specifically disappointed you? Please be detailed.' — Required on this page.

Step 4: Add a 'Single Select' question: 'Which aspect needs the most improvement?' — Options: Poor Organization, Irrelevant Content, Technical Issues, Venue Problems, Speaker Quality, Timing.

Page 4: Suggestions & File Upload

Step 1: Add a 'Long Text' question: 'Any suggestions for future events?'

Step 2: Add a 'File Upload' question: 'Attach a photo from the event (optional)' — set Max files: 1, Accepted types: image/*.

Step 3: Add a 'Single Select' question: 'Would you attend a similar event again?' — Options: Definitely, Probably, Probably Not, Definitely Not.

Step 4: Save the form and click Preview to test the conditional logic — submit with a rating of 1 and verify Page 3 appears; submit with rating 4 and verify Page 3 is skipped.

Part C: Configure Webhook to n8n

Step 1: In n8n (<http://localhost:5678>), click '+ New Workflow'.

Step 2: Add a 'Webhook' trigger node. Set Method: POST. Copy the webhook URL shown.

Step 3: Save the webhook URL — it looks like: <http://localhost:5678/webhook/xxxxxxxxx>

Step 4: In Formbricks, go to your survey → Settings → Integrations → Webhook.

Step 5: Click 'Add Webhook'. Paste the n8n webhook URL. Select trigger: 'On submission'. Save.

Step 6: Submit a test response in Formbricks. Return to n8n — you should see 'Webhook received' with the form data.

Part D: Build the n8n Workflow

In n8n, extend the workflow to process and store the form response.

Step 1: After the Webhook trigger, add a 'Set' node. Map incoming fields to cleaner variable names: name, usn, branch, rating, suggestions.

Step 2: Add an 'HTTP Request' node — this will POST the data to NocoDB. Configure:

- Method: POST
- URL: `http://nocodb:8080/api/v1/db/data/noco/{your-project-id}/Feedback`
- Authentication: Header Auth — name: xc-token, value: {your NocoDB API token}
- Body: JSON with the mapped fields from the Set node

Step 3: Add an 'IF' node after Set: Condition: `{{$.rating}}` less than 3.

Step 4: On the 'false' branch (rating $\geq$ 3): add a 'No Operation' node labeled 'Positive — no alert'.

Step 5: On the 'true' branch (rating $<$ 3): add another 'HTTP Request' node that POSTs a separate record to a 'NegativeFeedback' table in NocoDB.

Step 6: Connect both branches to a final 'Respond to Webhook' node that returns status: success.

Step 7: Save and Activate the workflow.

TIP: Create the 'Feedback' and 'NegativeFeedback' tables in NocoDB first with matching fields, then return to n8n.

Part E: Build the Thank-You Landing Page in GrapesJS

Step 1: Open GrapesJS at <https://grapesjs.com/demo.html> in your browser (no install needed).

Step 2: Delete all default blocks. Build a simple thank-you page with:

- A full-width hero section with background color #1A3C5E
- A centered heading: 'Thank You for Your Feedback!' in white
- A subheading: 'Your response has been recorded. See you at the next event!'
- A button styled in amber (#E8A020) linking back to the college website
- A footer with college name and current year

Step 3: Use the Style Manager to set: font-family: Arial, font-size: 18px on body.

Step 4: Check the responsive view (tablet and mobile icons at top) — ensure the heading scales properly.

Step 5: Click 'Export Code' → copy the HTML content.

Step 6: Create a new GitHub repository named 'lcnc-thankyou'. Create an index.html file and paste the exported code.

Step 7: Go to repo Settings → Pages → Source: main branch → /root. Save. GitHub will provide a public URL in ~2 minutes.

Step 8: Return to Formbricks survey settings → Ending → Redirect URL. Paste the GitHub Pages URL.

End-to-End Test

Submit the form with a rating of 2 (to trigger the negative feedback branch). Verify:

- n8n workflow executes all nodes without error

- A record appears in the Feedback table in NocoDB
- A record appears in the NegativeFeedback table in NocoDB
- Browser redirects to your GitHub Pages thank-you page after submission

OUTCOME: Students have built a complete form-to-automation-to-database-to-page pipeline. A form submission triggers an n8n workflow, stores data in NocoDB via API, applies conditional routing, and delivers the user to a publicly deployed custom landing page.

Evaluation Rubric

Criterion	Marks	Description
Form has 4 pages with all specified questions	15	All question types used correctly; required fields marked
Conditional logic on Page 3 works correctly	20	Page 3 shown for rating < 3 and hidden for rating >= 3
Webhook triggers n8n workflow successfully	15	Test submission visible in n8n execution log
n8n workflow stores data in NocoDB	20	Records appear in Feedback table with correct field mapping
IF branch routes negative feedback correctly	15	NegativeFeedback table receives records only for low ratings
Thank-you page deployed and redirect working	15	GitHub Pages URL accessible; form redirects after submission

LAB 3 Full CRUD Application with Role-Based UI

Tools: Appsmith · NocoDB

Duration	3 hours
Tools	Appsmith (port 8081), NocoDB (port 8080)
Prerequisite	Lab 1 completed; NocoDB API token available
Difficulty	Medium

Objective

Build a multi-screen Leave Management System in Appsmith that supports two roles (Student and Admin) and performs all four CRUD operations against the NocoDB backend using REST API calls.

Theory

Appsmith is a low-code platform for building internal tools. It provides a drag-and-drop canvas where you add widgets (tables, forms, buttons, modals) and wire them to data sources (REST APIs, databases, GraphQL). JavaScript expressions control widget visibility, data transformation, and conditional logic — giving you full programmatic control without a traditional frontend framework.

Part A: Appsmith Setup & NocoDB Data Source

- Step 1:** Open Appsmith at <http://localhost:8081>. Click 'Sign Up' and create an account.
- Step 2:** Click 'Create New' → 'Application'. Name it 'Leave Management System'.
- Step 3:** In the left sidebar, click 'Data Sources' → '+ New Datasource' → 'REST API'.
- Step 4:** Name the datasource 'NocoDB'. Set Base URL: <http://nocodb:8080>
- Step 5:** Add a Header: Key = xc-token, Value = {your NocoDB API token from Lab 1}.
- Step 6:** Click 'Test' — you should see 'Connection successful'. Save the datasource.

Part B: Create the NocoDB Leave Table

Switch to NocoDB and create the data table before building the UI.

- Step 1:** In NocoDB, open your project → Add new table 'LeaveRequests'.
- Step 2:** Add these fields:

Field Name	Type & Config
Student Name	Single line text — Required
USN	Single line text — Required
Branch	Single Select — CSE, ISE, ECE, ME, CV, DS

From Date	Date — Required
To Date	Date — Required
Reason	Long text — Required
Leave Type	Single Select — Medical, Personal, Academic, Family
Status	Single Select — Pending, Approved, Rejected — Default: Pending
Admin Remarks	Long text
Submitted At	Created time (auto)

Step 3: Add 5 sample records manually so Appsmith has data to display.

Part C: Build the Student View

Page 1 of the app is the Student interface — submit new requests and view own submissions.

Step 1: Create API Queries

Step 1: In Appsmith, click 'Queries/JS' → '+ New Query' → select NocoDB datasource.

Step 2: Create query 'getLeaveRequests': GET /api/v1/db/data/noco/{projectId}/LeaveRequests

Step 3: Add query params: where=(USN,eq,{{studentUSN.text}}), limit=50

Step 4: Create query 'createLeaveRequest': POST /api/v1/db/data/noco/{projectId}/LeaveRequests with body:

```
{
  "Student Name": "{{nameInput.text}}",
  "USN": "{{usnInput.text}}",
  "Branch":
    "{{branchSelect.selectedOptionValue}}",
  "From Date":
    "{{fromDatePicker.formattedDate}}",
  "To Date": "{{toDatePicker.formattedDate}}",
  "Reason": "{{reasonInput.text}}",
  "Leave Type":
    "{{leaveTypeSelect.selectedOptionValue}}"
}
```

Step 2: Build the Student Dashboard

Step 1: Drag a 'Table' widget onto the canvas. Set 'Table data' to: {{getLeaveRequests.data.list}}

Step 2: Configure visible columns: Student Name, From Date, To Date, Leave Type, Status. Hide others.

Step 3: Add a 'Status Badge' column — use a computed column with expression:

```
{{currentRow.Status === 'Approved' ? '✅ Approved' : currentRow.Status === 'Rejected' ?
'❌ Rejected' : '⌚ Pending'}}
```

Step 4: Drag a 'Button' widget labeled 'New Leave Request'. Set onClick to open a Modal widget.

Step 5: Inside the modal, add these input widgets:

- Text input for Name, USN
- Select widget for Branch with options: CSE, ISE, ECE, ME, CV, DS
- Date picker for From Date and To Date
- Text area for Reason
- Select widget for Leave Type

Step 6: Add a 'Submit' button inside the modal. Set onClick to run createLeaveRequest query, then close modal, then run getLeaveRequests.

Step 7: Add input validation using Appsmith's 'Valid' property: set USN input valid condition: `{{/^([0-9][A-Z]{2}[0-9]{2}[A-Z]{2}[0-9]{3})$/ .test(usnInput.text)}}`

Step 8: Set Submit button 'Disabled' property to: `{{!createLeaveRequest.isDataAvailable || nameInput.text === "" || usnInput.text === ""}}`

Part D: Build the Admin View

Page 2 is the Admin dashboard — sees all requests, can approve or reject with remarks.

Step 1: Click '+ Page' in Appsmith sidebar. Name it 'Admin Dashboard'.

Step 2: Create query 'getAllRequests': GET `/api/v1/db/data/noco/{projectId}/LeaveRequests` — no USN filter.

Step 3: Create query 'updateStatus': PATCH `/api/v1/db/data/noco/{projectId}/LeaveRequests/{selectedRow.Id}` with body:

```
{  "Status": "{{statusRadio.selectedValue}}",  "Admin Remarks":
  "{{remarksInput.text}}" }
```

Step 4: Add a Table widget showing all requests. Add row selection (click a row to select).

Step 5: Add a 'Detail Panel' container (right side) that becomes visible when a row is selected: `{{!!allRequestsTable.selectedRow}}`

Step 6: Inside the Detail Panel, add:

- Text widgets showing selected row details (`{{allRequestsTable.selectedRow.Student_Name}}` etc.)
- Radio group widget for Status: options Approved, Rejected
- Text area for Admin Remarks
- 'Update' button that runs `updateStatus` then `getAllRequests`

Step 7: Style the Status column with conditional cell background:

```
{{currentRow.Status === 'Approved' ? '#E8F5EE' : currentRow.Status === 'Rejected' ?
'#FDE8E8' : '#FFF8E6'}}
```

Part E: Role Selector on Home Page

Step 1: Create a Page 1 'Home' with two large buttons: 'I am a Student' and 'I am an Admin'.

Step 2: Set onClick of Student button: `navigateTo('Student View')`.

Step 3: Set onClick of Admin button: show a password modal. On password match (admin123), `navigateTo('Admin Dashboard')`. Otherwise show error toast.

Part F: Delete Functionality

Step 1: In the Student view, add a 'Delete' icon button in the table row (visible only for Pending requests: `{{currentRow.Status === 'Pending'}}`).

Step 2: Create query 'deleteRequest': DELETE `/api/v1/db/data/noco/{projectId}/LeaveRequests/{leaveTable.selectedRow.Id}`

Step 3: Set Delete button onClick to: `showAlert('Are you sure?', 'warning')` then run `deleteRequest`, then `getLeaveRequests`.

OUTCOME: Students have built a full multi-role CRUD application without writing a single line of backend code. The app supports four HTTP methods (GET, POST, PATCH, DELETE), conditional UI rendering, form validation, and role-based navigation.

Evaluation Rubric

Criterion	Marks	Description
NocoDB datasource connected in Appsmith	10	Datasource test passes; API token configured
Student view: table displays filtered records	15	Only shows records matching entered USN
Student view: new request form with validation	20	All fields present; USN format validated; Submit disabled when invalid
Admin view: all records visible with approve/reject	25	updateStatus query fires correctly; NocoDB record updated
Role selector with password protection on Admin	15	Correct password navigates; wrong password shows error
Delete works and only for Pending requests	15	Delete button hidden for Approved/Rejected; record removed from NocoDB

LAB 4 Multi-Step Workflow Automation with Error Handling

Tools: n8n · NocoDB · SMTP

Duration	3 hours
Tools	n8n (port 5678), NocoDB (port 8080), SMTP (use Mailpit for local email testing)
Prerequisite	Labs 1 & 3 completed
Difficulty	Medium

Objective

Build a Student Onboarding Automation workflow in n8n that triggers on new NocoDB records, validates data, sends personalized emails, handles errors gracefully, writes structured logs, and runs a scheduled daily summary — all without writing a backend service.

Theory

n8n is a workflow automation platform that chains 'nodes' (trigger + action steps) into executable pipelines. It supports event-driven triggers (webhooks, database polling, schedules), conditional branching (IF nodes), data transformation (Set, Function nodes), error handling (Error Trigger, Catch nodes), and hundreds of integrations. Each node receives input items and passes output items to the next node.

Part A: Set Up Mailpit (Local Email Testing)

Mailpit is a local SMTP server that captures all emails without sending them — perfect for lab testing.

Step 1: Add Mailpit to your stack by running:

```
docker run -d --name lcnc_mailpit --network lcnc_lcnc_network -p 1025:1025 -p 8025:8025 axllent/mailpit
```

Step 2: Open <http://localhost:8025> — this is the Mailpit inbox where you will see captured emails.

Step 3: In n8n, configure SMTP credential: host=lcnc_mailpit, port=1025, no auth, no SSL.

Part B: Create the Students Table in NocoDB

Step 1: In NocoDB, add a new table 'Students' with these fields:

Field Name	Type & Config
Full Name	Single line text — Required
USN	Single line text — Required

Email	Email — Required
Branch	Single Select — CSE, ISE, ECE, ME, CV, DS
Semester	Number — 1 to 8
Phone	Phone number
Onboarding Status	Single Select — Pending, Emailed, Failed — Default: Pending
Created At	Created time (auto)

Part C: Build the Onboarding Workflow

Step 1: In n8n, create a new workflow named 'Student Onboarding Automation'.

Node 1: NocoDB Trigger (Poll for new Students)

Step 1: Add trigger: 'Schedule Trigger' — interval: every 1 minute (for testing).

Step 2: Add an 'HTTP Request' node to fetch new students:

- GET `http://lcnc_nocodb:8080/api/v1/db/data/noco/{projectId}/Students`
- Query param: `where=(Onboarding_Status,eq,Pending), limit=10`
- Header: `xc-token: {your token}`

Step 3: Add a 'Split In Batches' node — set batch size to 1. This processes each student individually.

Node 2: Validate Data

Step 1: Add an 'IF' node after the batch splitter.

Step 2: Set condition: `{{ $json['Email'] }}` is not empty AND `{{ $json['Full Name'] }}` is not empty AND `{{ $json['USN'] }}` is not empty.

Step 3: True branch = valid data. False branch = invalid, handle separately.

Node 3a: Send Welcome Email (True Branch)

Step 1: Add 'Send Email' node on the True branch. Configure:

- From: `noreply@lcncclab.local`
- To: `{{ $json.Email }}`
- Subject: Welcome to the Portal, `{{ $json['Full Name'] }}`!

HTML body (paste into the HTML editor in n8n):

```
<html><body style="font-family:Arial;padding:20px;">    <h2
style="color:#1A3C5E;">Welcome, {{ $json['Full Name'] }}!</h2>    <p>Your student account
has been successfully registered.</p>    <table>
<tr><td><b>USN:</b></td><td>{{ $json['USN'] }}</td></tr>
<tr><td><b>Branch:</b></td><td>{{ $json['Branch'] }}</td></tr>
<tr><td><b>Semester:</b></td><td>{{ $json['Semester'] }}</td></tr>    </table>    <p>Please
log in at http://localhost:8081 to access your dashboard.</p> </body></html>
```

Node 4a: Update Status to 'Emailed' (after successful email)

Step 1: Add 'HTTP Request' node:

- PATCH `http://lcnc_nocodb:8080/api/v1/db/data/noco/{projectId}/Students/{{$.Id}}`
- Body: `{ Onboarding_Status: 'Emailed' }`

Node 4b: Write Error Log (False branch — invalid data)

Step 1: On the False branch, add 'HTTP Request' node to POST to `automation_logs` table:

```
{
  "event_type": "student_onboarding_validation_failed",
  "status": "error",
  "message": "Missing required fields for student ID {{$.Id}}",
  "payload": {{$.json}}
}
```

Step 2: Add another HTTP Request to PATCH the student status to 'Failed'.

Node 5: Global Error Handler

Step 1: Add an 'Error Trigger' node (separate workflow entry point for errors).

Step 2: Connect it to a 'Send Email' node that emails the admin: `admin@lcncfab.local` with the error details.

Step 3: Enable 'Error Workflow' in the main workflow settings → select this error workflow.

Part D: Scheduled Daily Summary

Step 1: Create a second workflow: 'Daily Summary Report'.

Step 2: Add 'Schedule Trigger' — CRON: `0 18 * * *` (runs at 6 PM every day).

Step 3: Add 'HTTP Request' to fetch all students where `Onboarding_Status = Emailed` and `Created_At` is today.

Step 4: Add 'Set' node to compute total count: `{{$.items().length}}`

Step 5: Add 'HTTP Request' to fetch Failed count similarly.

Step 6: Add 'Send Email' to admin with an HTML table summarizing today's onboarding: total students, emailed successfully, failed.

Part E: End-to-End Test

Step 1: Add a new Student record in NocoDB with Status = Pending and a valid email.

Step 2: Within 1 minute, check Mailpit (`http://localhost:8025`) — the welcome email should appear.

Step 3: Check the Student record in NocoDB — Status should now be 'Emailed'.

Step 4: Add a Student record with an empty Email field. Verify it goes to Failed status and logs appear in `automation_logs`.

Step 5: Manually trigger the Daily Summary workflow and verify the summary email in Mailpit.

OUTCOME: Students have built a production-quality automation pipeline with polling triggers, field validation, conditional routing, HTML email generation, status updates, error logging, and scheduled reporting — all configured visually in n8n.

Evaluation Rubric

Criterion	Marks	Description
Workflow triggers and fetches Pending students	15	Polling triggers, HTTP request returns records correctly
Validation IF node correctly routes valid/invalid	20	Test both branches with deliberate invalid records
Welcome email sent with correct personalized content	20	Email visible in Mailpit with all student fields populated
Status updated to Emailed/Failed in NocoDB	15	Records reflect correct status after workflow execution
Error log written to automation_logs table	15	JSON payload visible in NocoDB automation_logs
Daily summary email contains correct counts	15	Summary email shows accurate totals for the day

LAB 5 IoT Data Pipeline & Live Dashboard

Tools: Node-RED · Node-RED Dashboard

Duration	3 hours
Tools	Node-RED (port 1880), Node-RED Dashboard (install via Palette Manager)
Prerequisite	Global setup completed
Difficulty	Medium

Objective

Model a realistic multi-sensor IoT system using Node-RED flow-based programming. Simulate three sensors, classify readings into alert levels, suppress duplicate alerts, log events to file, and display a live dashboard with gauges, charts, and status indicators.

Theory

Node-RED is a flow-based programming tool originally designed for IoT at IBM. 'Flows' are visual pipelines where data (called 'messages' or msg objects) flows between nodes. Each node processes the msg, transforms it, and passes it to the next. This model mirrors real IoT architectures: sensor → gateway → processing → storage/display.

Part A: Install Dashboard Nodes

- Step 1:** Open Node-RED at <http://localhost:1880>.
- Step 2:** Click the hamburger menu (top right) → 'Manage palette'.
- Step 3:** Go to 'Install' tab. Search 'node-red-dashboard'. Click Install.
- Step 4:** Wait for installation (~1-2 minutes). You will see dashboard nodes appear in the left palette.
- Step 5:** Also install: 'node-red-node-tail' for file operations.

Part B: Sensor Simulation Flows

Create three separate sensor injection flows.

Sensor 1: Temperature

Step 1: Drag an 'Inject' node onto the canvas. Set repeat: interval of 3 seconds. Payload: empty (we generate in Function).

Step 2: Add a 'Function' node. Double-click and paste this code:

```
// Simulate temperature sensor with occasional spikes var base = 22 + Math.random() * 8;
// Normal: 22-30°C var spike = Math.random() > 0.85 ? 15 : 0; // 15% chance of spike
```



```
msg.payload = {      sensor: "temperature",      value: parseFloat((base +
spike).toFixed(1)),      unit: "°C",      timestamp: new Date().toISOString() }; return
msg;
```

Step 3: Add another 'Function' node to classify the reading:

```
var v = msg.payload.value; if (v < 28) {      msg.payload.level = "Normal";
msg.payload.color = "green"; } else if (v < 35) {      msg.payload.level = "Warning";
msg.payload.color = "yellow"; } else {      msg.payload.level = "Critical";
msg.payload.color = "red"; } return msg;
```

Sensor 2: Humidity

Step 1: Add another Inject node — repeat every 5 seconds.

Step 2: Add a Function node:

```
msg.payload = {      sensor: "humidity",      value: parseFloat((40 + Math.random() *
50).toFixed(1)),      unit: "%",      timestamp: new Date().toISOString() }; return msg;
```

Step 3: Add a classifier Function node:

```
var v = msg.payload.value; msg.payload.level = v < 30 ? "Low" : v > 80 ? "High" :
"Normal"; msg.payload.color = (msg.payload.level === "Normal") ? "green" : "yellow";
return msg;
```

Sensor 3: Door Contact Switch

Step 1: Add Inject node — repeat every 7 seconds.

Step 2: Function node:

```
msg.payload = {      sensor: "door",      value: Math.random() > 0.7 ? "OPEN" : "CLOSED",
timestamp: new Date().toISOString() }; msg.payload.level = msg.payload.value === "OPEN"
? "Warning" : "Normal"; msg.payload.color = msg.payload.value === "OPEN" ? "yellow" :
"green"; return msg;
```

Part C: Alert Rate Limiter

Suppress repeated alerts so the same alert does not fire more than once per 60 seconds.

Step 1: For each sensor, route Warning/Critical level readings through a 'Delay' node in 'Rate Limit' mode:

- Action: Drop intermediate messages
- Rate: 1 message per 60 seconds
- Scope: per topic (set msg.topic = msg.payload.sensor + '-alert' in a Function node before the Delay)

Step 2: Connect the output of the rate limiter to an 'Alert Log' write node (covered in Part D).

Part D: File Logging

Step 1: Add a 'Function' node that formats the alert as a log line:

```
msg.payload = new Date().toISOString() + " | " +      msg.payload.sensor.toUpperCase() +
" | " +      msg.payload.level + " | " +      msg.payload.value + " " + (msg.payload.unit
|| "") + " "; return msg;
```

Step 2: Add a 'File' node (from node-red-node-file). Set filename: /data/sensor_alerts.log. Action: Append to file.

Part E: Build the Live Dashboard

Step 1: In Node-RED, go to Dashboard tab (right side panel) → click 'Add Tab'. Name it 'IoT Monitor'.

Step 2: Add 3 Groups under the tab: Temperature, Humidity, Door.

Temperature Widgets

Step 1: Add 'Gauge' node (dashboard). Set Group: Temperature, Label: Temp (°C), Min: 0, Max: 60. Connect from temperature sensor flow.

Step 2: Add 'Chart' node. Type: Line chart. Group: Temperature. Label: Temperature Trend. X-axis: last 10 minutes. Connect from sensor flow.

Step 3: Add 'Text' node. Group: Temperature. Label: Alert Level. Value: `{{msg.payload.level}}`. Connect after classifier.

Humidity Widgets

Step 1: Add 'Gauge' node: Group: Humidity, Min: 0, Max: 100, Sectors: 0-30 red, 30-80 green, 80-100 yellow.

Step 2: Add 'Chart' node: Line chart for humidity trend.

Door Widgets

Step 1: Add 'Text' node showing door state (OPEN/CLOSED).

Step 2: Add 'LED indicator' using a Template node:

```
<div style="width:40px;height:40px;border-radius:50%;
background:{{msg.payload.value==='OPEN'?'#ff6b35':'#2ecc71'}}; margin:auto;box-shadow:0
0 15px {{msg.payload.value==='OPEN'?'#ff6b35':'#2ecc71'}}"> </div>
```

Alert Feed

Step 1: Add a 4th Group: 'System Alerts'.

Step 2: Merge all three rate-limited alert streams using a 'Join' or simply connecting all three to one 'Text' widget set to display last 5 messages.

Step 3: Add a notification node: Dashboard → 'Notification'. Connect rate-limited alerts. Set: position: bottom right.

Part F: Failure Simulation

Step 1: Add a 'Catch' node. Connect it to a 'Function' node that sets `msg.payload = 'SENSOR FAILURE: ' + error.message`.

Step 2: Connect to a Dashboard 'Text' widget in a 'System Status' group with red background.

Step 3: To test: deliberately introduce a syntax error in one Function node, deploy, and observe the catch node fires.

Access the Dashboard

Open <http://localhost:1880/ui> — the live dashboard updates in real time. Share this URL to view on any device on the same network.

OUTCOME: Students have built a realistic multi-sensor IoT data pipeline with simulated sensors, multi-level classification, rate-limited alerting, file logging, failure recovery, and a live multi-widget dashboard — directly applicable to real embedded engineering problems.

Evaluation Rubric

Criterion	Marks	Description
Three sensor flows generate data every few seconds	15	Dashboard shows live updating values for all three sensors
Classification correctly assigns levels and colors	15	Test values in each zone (Normal/Warning/Critical) route correctly
Rate limiter suppresses repeat alerts within 60s	20	Only one alert visible per sensor per minute under steady state
File log captures alerts with timestamp	15	sensor_alerts.log file contains formatted entries
Dashboard has all required widgets (gauge, chart, LED, feed)	20	Dashboard UI complete with all 4 groups visible
Catch node handles injected errors gracefully	15	System Status widget shows error without crashing the flow

LAB 6 Zero-Code Analytics Dashboard with Data Storytelling

Tools: Metabase · PostgreSQL

Duration	3 hours
Tools	Metabase (port 3001), PostgreSQL (pre-loaded orders dataset)
Prerequisite	Global setup (init-db.sql creates sample data automatically)
Difficulty	Medium

Objective

Build a comprehensive business intelligence dashboard in Metabase over a 500-row e-commerce dataset. Create 7 different question types, assemble a filterable dashboard, add narrative text cards, and embed one chart externally.

Theory

Business Intelligence (BI) tools help organizations understand their data visually. Metabase's 'Question' builder allows non-technical analysts to explore databases using a visual query interface — selecting tables, adding filters, choosing groupings, and picking chart types — without writing SQL. The result is a reproducible, shareable artifact that updates automatically when underlying data changes.

Part A: Metabase Initial Setup

Step 1: Open Metabase at <http://localhost:3001>.

Step 2: Complete the setup wizard. On 'Add your data' step, choose PostgreSQL:

- Host: postgres
- Port: 5432
- Database name: lcncclab
- Username: lcncadmin
- Password: lcncpass123

Step 3: Click 'Next' — Metabase will sync the database schema.

Step 4: Finish setup. You will land on the Metabase home screen.

Step 5: Click 'Browse Data' → 'lcncclab' → 'Public' → 'Orders'. Verify 500 rows are visible.

Part B: Build 7 Questions (Charts)

Click '+ New' → 'Question' for each. Use 'Simple Question' or 'Custom Question' mode.

Q1: Total Revenue

Step 1: Pick 'Orders'. Click 'Summarize'. Choose 'Sum of Total Amount'. Remove all groupings.

Step 2: Visualization: Big Number. Label: Total Revenue. Format: Currency, prefix ₹.

Step 3: Save as 'KPI: Total Revenue'.

Q2: Monthly Revenue Trend

Step 1: Pick 'Orders'. Summarize: Sum of Total Amount, grouped by Order Date (month).

Step 2: Visualization: Line chart. X-axis: Order Date, Y-axis: Sum of Total Amount.

Step 3: Save as 'Monthly Revenue Trend'.

Q3: Revenue by Region

Step 1: Pick 'Orders'. Summarize: Sum of Total Amount, grouped by Region.

Step 2: Visualization: Bar chart, sorted descending.

Step 3: Save as 'Revenue by Region'.

Q4: Top 10 Products by Revenue

Step 1: Pick 'Orders'. Filter: Is Returned = False. Summarize: Sum of Total Amount, grouped by Product Name. Sort: descending. Limit: 10.

Step 2: Visualization: Horizontal bar chart.

Step 3: Save as 'Top 10 Products'.

Q5: Orders by Payment Method

Step 1: Pick 'Orders'. Summarize: Count of rows, grouped by Payment Method.

Step 2: Visualization: Pie chart. Enable percentage labels.

Step 3: Save as 'Payment Method Distribution'.

Q6: Return Rate by Category

Step 1: Pick 'Orders'. Use Custom Question. Group by Category. Add two metrics:

- Count of rows (label: Total Orders)
- Count of rows filtered by Is Returned = True (label: Returns)

Step 2: Visualization: Combo chart (bars for Total, line for Returns).

Step 3: Save as 'Return Rate by Category'.

Q7: Average Order Value by Month

Step 1: Pick 'Orders'. Summarize: Average of Total Amount, grouped by Order Date (month).

Step 2: Visualization: Area chart.

Step 3: Save as 'Average Order Value Trend'.

Part C: Assemble the Dashboard

Step 1: Click '+ New' → 'Dashboard'. Name it 'E-Commerce Performance Report'.

Step 2: Click '+ Add a question' — add all 7 questions.

Step 3: Arrange them in a logical layout:

- Row 1: KPI Total Revenue (small) + two more KPIs
- Row 2: Monthly Revenue Trend (full width)
- Row 3: Revenue by Region (left) + Payment Distribution (right)
- Row 4: Top 10 Products (left) + Return Rate by Category (right)
- Row 5: Average Order Value Trend (full width)

Part D: Add Text Cards (Data Storytelling)

Step 1: Click '+ Add a text card' between Row 1 and Row 2.

Step 2: Write a 2-3 sentence narrative: 'Revenue has grown steadily over the past 12 months with a peak in Q4. The South region consistently outperforms other regions. UPI dominates as the preferred payment method.'

Step 3: Add another text card above the Return Rate chart: 'The Electronics category shows the highest return rate at 18%, suggesting a product quality or description mismatch that requires attention.'

Part E: Add Filters

Step 1: Click the filter icon in the dashboard. Add 'Date Range' filter — link it to Order Date column in all date-based questions.

Step 2: Add 'Category' dropdown filter — link to Category column in Q4 and Q6.

Step 3: Add 'Region' dropdown filter — link to Region column in Q3.

Step 4: Test: select a date range of last 6 months — all linked charts should update simultaneously.

Part F: Embed a Chart

Step 1: Open the 'Revenue by Region' question. Click Share → 'Embed this question in an application'.

Step 2: Enable embedding. Copy the iframe code.

Step 3: Open your GrapesJS thank-you page (from Lab 2) in the editor. Add an HTML block and paste the iframe code.

Step 4: Preview the page — the chart renders embedded.

OUTCOME: Students have gone beyond chart generation to structured analytical thinking — building 7 varied visualizations, a multi-filter interactive dashboard, narrative text storytelling, and chart embedding. They understand how non-technical stakeholders consume data insights.

Evaluation Rubric

Criterion	Marks	Description
All 7 questions built with correct chart types	25	Each chart displays correct data with appropriate visualization
Dashboard assembled in logical layout	10	Questions arranged sensibly; dashboard loads without errors
2 text cards with meaningful analysis narrative	20	Narratives reference specific numbers from the data
3 filters applied and linked correctly	25	Filters dynamically update all linked charts
Chart embedded in external page	10	iframe renders Metabase chart in GrapesJS page
Data storytelling quality	10	Narrative accurately reflects the dataset patterns

LAB 7 Responsive Website with Component System

Tools: GrapesJS · GitHub Pages

Duration	3 hours
Tools	GrapesJS (browser-based), GitHub (free account required)
Prerequisite	GitHub account created
Difficulty	Medium

Objective

Build a 4-page responsive product website using GrapesJS. Establish a reusable component library, define a consistent design token system, create a CSS animation block, and deploy all pages to GitHub Pages with functional inter-page navigation.

Theory

Component-based design is the foundation of modern frontend development. Rather than building unique one-off layouts, designers create a library of reusable components (nav, hero, card, footer) that share consistent styling tokens (colors, fonts, spacing). GrapesJS formalizes this through its Component system and Style Manager — the same mental model used in Figma, React, and design systems like Material UI.

Design Token System (Use Throughout All Pages)

Token	Value
Primary Color	#1A3C5E (Deep Navy)
Accent Color	#E8A020 (Amber)
Background	#F4F7FB (Off-white)
Surface	#FFFFFF (White)
Text Primary	#1A1A2E (Near black)
Text Muted	#6B7280 (Gray)
Font Display	Georgia or Playfair Display
Font Body	Arial or Inter
Border Radius	8px for cards, 4px for buttons
Shadow	0 2px 12px rgba(0,0,0,0.08)
Max Width	1140px (centered container)

Section Padding

80px vertical, 20px horizontal

Product Concept

You will build a website for 'DevKit Pro' — a fictional software developer toolkit. The 4 pages are: Landing, Features, Pricing, Contact.

Part A: Set Up GrapesJS

Step 1: Open <https://grapesjs.com/demo.html> in your browser. This is a full GrapesJS editor — no installation.

Step 2: Click the settings icon (top right of canvas) → toggle 'Device Manager' to show mobile/tablet views.

Step 3: Familiarize yourself with: Block Manager (left), Style Manager (right), Layers panel (layers icon), Component tree.

Part B: Build Reusable Components

Build these 5 components before building any pages. GrapesJS lets you right-click any element → 'Save as Component'.

Component 1: Navigation Bar

Step 1: Add a 1-column layout block. Set background: #1A3C5E, padding: 0 40px.

Step 2: Add a div inside with display:flex, justify-content:space-between, align-items:center, max-width:1140px, margin:0 auto.

Step 3: Left side: Text element 'DevKit Pro' — font: Georgia, size: 24px, color: white, font-weight: bold.

Step 4: Right side: 4 link elements — Landing, Features, Pricing, Contact — color: #E8A020, margin: 0 16px.

Step 5: Right-click the whole navbar div → 'Save as Block'. Name it 'NavBar'.

Component 2: Hero Section

Step 1: Full-width section with background: linear-gradient(135deg, #1A3C5E, #2A6099), padding: 100px 40px, text-align: center.

Step 2: Add heading: 'Build Better Software Faster' — font: Georgia, size: 48px, color: white.

Step 3: Add subheading paragraph — font: Arial, size: 20px, color: rgba(255,255,255,0.85), max-width: 600px, margin: 0 auto.

Step 4: Add a button: 'Get Started Free' — background: #E8A020, color: white, padding: 16px 36px, border-radius: 4px, border: none.

Step 5: Save as Block: 'HeroSection'.

Component 3: Feature Card

Step 1: A white card div: border-radius: 8px, box-shadow: 0 2px 12px rgba(0,0,0,0.08), padding: 32px, width: 300px.

Step 2: Inside: an icon placeholder (colored square #E8A020, 48x48), heading (font:Arial, 18px, bold, #1A3C5E), body text (Arial, 15px, #6B7280).

Step 3: Save as Block: 'FeatureCard'.

Component 4: Pricing Card

Step 1: Card with border: 2px solid #E8A020, border-radius: 8px, padding: 40px, text-align: center.

Step 2: Inside: Plan name (bold, 20px), Price (48px, #1A3C5E), '/month' (14px, gray), feature list (text, 15px), CTA button.

Step 3: Save as Block: 'PricingCard'.

Component 5: Footer

Step 1: Dark section: background: #0F1F30, padding: 40px, color: white.

Step 2: Three-column layout: About text, Quick Links list, Contact info.

Step 3: Bottom bar with copyright text and current year.

Step 4: Save as Block: 'Footer'.

Part C: Build the 4 Pages

For each page: place NavBar, the page-specific content, then Footer. Set the nav link hrefs to the other pages.

Page 1: Landing (index.html)

Step 1: Place NavBar + HeroSection + a 3-column row of 3 FeatureCards + a 2-column 'About' section (text left, image placeholder right) + Footer.

Step 2: Section padding between each block: 80px.

Page 2: Features (features.html)

Step 1: Place NavBar + a page header ('All Features', centered, navy) + a 3x3 grid of FeatureCards (9 total) + a comparison table (simple HTML table styled with tokens) + Footer.

Page 3: Pricing (pricing.html)

Step 1: Place NavBar + page header + 3 PricingCards (Free, Pro, Enterprise) in a row + an FAQ section (5 questions, accordion style using GrapesJS HTML blocks) + Footer.

Page 4: Contact (contact.html)

Step 1: Place NavBar + page header + a 2-column layout: left column has a contact form (Name, Email, Message, Submit button); right column has address/phone/email text + a map placeholder div.

Step 2: Style the Submit button: background #E8A020, hover effect via :hover CSS in style manager.

Part D: CSS Animation Block

Step 1: On the Landing page, click the 'About' section heading. In Style Manager, go to custom CSS.

Step 2: Add a CSS animation to the Hero button — it should pulse subtly:

```
@keyframes pulse { 0% { box-shadow: 0 0 0 0 rgba(232,160,32,0.6); } 70% { box-shadow: 0 0 0 12px rgba(232,160,32,0); } 100% { box-shadow: 0 0 0 0 rgba(232,160,32,0); } } .cta-button { animation: pulse 2s infinite; }
```

Step 3: Add class 'cta-button' to the Hero button element.

Part E: Responsive Breakpoints

Step 1: Switch to Tablet view (768px) in device manager. Adjust: make 3-column grids become 2-column using CSS: grid-template-columns: repeat(2, 1fr).

Step 2: Switch to Mobile view (375px). Make all multi-column layouts single column. Reduce font sizes (heading from 48px to 28px). Ensure buttons are full width.

Step 3: Test all 4 pages in all 3 viewports before export.

Part F: Export & Deploy to GitHub Pages

Step 1: For each page: click 'Export Code' → copy HTML → create the corresponding file (index.html, features.html, pricing.html, contact.html) in your GitHub repo.

Step 2: Update all nav link hrefs to relative paths between pages.

Step 3: Enable GitHub Pages (Settings → Pages → Source: main branch).

Step 4: Test all 4 pages live and verify all internal navigation links work.

OUTCOME: Students have practiced component-based design thinking, built a consistent design token system, mastered responsive breakpoints, added a CSS animation, and deployed a multi-page website — skills directly transferable to React, Vue, or any component-based frontend framework.

Evaluation Rubric

Criterion	Marks	Description
Design token system consistently applied across all 4 pages	20	Colors, fonts, spacing match defined tokens on every page
All 5 reusable components saved and used correctly	20	Components visible in Block Manager; NavBar and Footer used on every page

Criterion	Marks	Description
All 4 pages complete with correct content	15	Each page has appropriate sections matching requirements
CSS animation implemented and visible	10	Pulse effect visible on CTA button in Landing page
Responsive: mobile, tablet, and desktop all work	20	No horizontal scroll; layouts adapt correctly at each breakpoint
GitHub Pages deployed with all 4 pages navigable	15	All inter-page links functional on live GitHub Pages URL

LAB 8 AI Chatbot with Document Q&A and Memory

Tools: Flowise · Ollama · Llama 3

Duration	3 hours
Tools	Flowise (port 3002), Ollama (port 11434), Appsmith (port 8081)
Prerequisite	Llama 3 model pulled in Ollama (done in global setup). Lab 3 Appsmith app available.
Difficulty	Medium

Objective

Build a Retrieval Augmented Generation (RAG) chatbot in Flowise that answers questions from a college PDF document, maintains conversation memory across turns, runs entirely offline using Ollama, and is embedded into the Appsmith Leave Management app from Lab 3.

Theory

RAG (Retrieval Augmented Generation) is the most practical pattern for grounding AI responses in real documents. The pipeline has three phases: (1) Ingestion — split the document into chunks and embed each chunk as a vector in a vector store. (2) Retrieval — when a user asks a question, embed the question and find the most semantically similar chunks. (3) Generation — send the retrieved chunks as context to the LLM along with the question. The LLM answers from the provided context, not from training data, dramatically reducing hallucination.

Flowise makes this entire pipeline visual — each phase is a node. Ollama provides the LLM and embedding models locally, requiring no internet or API keys after initial model download.

Part A: Flowise Setup

Step 1: Open Flowise at <http://localhost:3002>. Login: admin / admin123.

Step 2: You will see the Flowise dashboard. Chatflows are visual LLM pipelines.

Step 3: Click '+ Add New' to create your first chatflow. Name it 'College Document Q&A'.

Part B: Prepare Your PDF Document

Step 1: Use your college's exam regulations PDF, VTU syllabus PDF, or hostel rules PDF (any multi-page official document).

Step 2: If unavailable, create a mock 'College Handbook' text file with 5 sections: Attendance Policy, Exam Rules, Fee Structure, Hostel Rules, Placement Eligibility. Save as PDF using LibreOffice.

Step 3: Keep the PDF accessible — you will upload it inside Flowise.

Part C: Build the RAG Pipeline in Flowise

The flow has 6 node types. Add them from the node panel on the right.

Node 1: Ollama Embeddings

Step 1: From 'Embeddings' category, drag 'Ollama Embeddings' node onto canvas.

Step 2: Base URL: `http://lcnc_ollama:11434`

Step 3: Model Name: `nomic-embed-text` (pull it first: `docker exec -it lcnc_ollama ollama pull nomic-embed-text`)

Node 2: In-Memory Vector Store

Step 1: From 'Vector Stores', drag 'In-Memory Vector Store'.

Step 2: Connect 'Ollama Embeddings' output → 'Embeddings' input of Vector Store.

Node 3: PDF File Loader

Step 1: From 'Document Loaders', drag 'PDF File' node.

Step 2: Click the upload icon in the node — upload your college handbook PDF.

Step 3: Set 'Text Splitter': Recursive Character Text Splitter, Chunk Size: 500, Chunk Overlap: 50.

Step 4: Connect PDF Loader output → 'Document' input of Vector Store.

Node 4: Ollama LLM

Step 1: From 'Chat Models', drag 'ChatOllama' node.

Step 2: Base URL: `http://lcnc_ollama:11434`

Step 3: Model Name: `llama3`

Step 4: Temperature: 0.2 (lower temperature means more factual, less creative answers).

Node 5: Conversational Retrieval QA Chain

Step 1: From 'Chains', drag 'Conversational Retrieval QA Chain'.

Step 2: Connect ChatOllama → 'Language Model' input.

Step 3: Connect Vector Store's 'Retriever' output → 'Vector Store Retriever' input.

Node 6: Buffer Memory

Step 1: From 'Memory', drag 'Buffer Memory' node.

Step 2: Set Memory Key: `chat_history`. Session ID: leave blank (auto per session).

Step 3: Connect Buffer Memory → 'Memory' input of the QA Chain.

Part D: Configure the System Prompt

Step 1: Click the QA Chain node → find 'System Message' or 'System Prompt' field.

Step 2: Enter this prompt:

You are a helpful college information assistant. Answer questions ONLY based on the provided document context. If the answer is not found in the context, respond with: "I could not find information about that in the college handbook. Please contact the college office for assistance." Keep answers concise (3-5 sentences maximum). Do not invent or assume information not in the document.

Part E: Test the Chatbot

Step 1: Click 'Save Chatflow' then the chat bubble icon (top right) to open the test chat.

Step 2: Test with at least 10 questions. Record which answer correctly (from document) vs. which says 'not found':

- Ask 3 questions whose answers ARE in the document.
- Ask 3 questions whose answers are NOT in the document (should trigger the not-found response).
- Ask 2 follow-up questions that require memory (e.g., 'What about the fee I asked earlier?').
- Ask 2 ambiguous questions to test the boundary of the system prompt constraint.

Step 3: Document your test results in a table with columns: Question, Expected, Actual, Correct Y/N.

Part F: Embed in Appsmith (Lab 3 App)

Step 1: In Flowise, click 'Configuration' → 'Embed'. Copy the iframe code snippet.

Step 2: Open your Appsmith Leave Management app from Lab 3.

Step 3: Add a new page named 'Help Assistant'.

Step 4: Drag an 'Iframe' widget onto the canvas.

Step 5: In the Iframe URL property, enter: `http://localhost:3002` (or the specific chatflow URL from Flowise embed settings).

Step 6: Resize the iframe to fill most of the screen. Add a heading above it: 'College Handbook Assistant (Powered by Llama 3)'.

Step 7: Add a link in the app's sidebar navigation pointing to the Help Assistant page.

OUTCOME: Students have implemented a complete offline RAG pipeline — PDF ingestion, vector embedding, semantic retrieval, LLM generation, and conversation memory — using a fully visual no-code tool. They understand the real mechanics behind enterprise AI assistants without writing a single line of Python.

Evaluation Rubric

Criterion	Marks	Description
All 6 Flowise nodes connected correctly	15	Flow saves without errors; no missing connections
PDF uploaded and embedded (vector store populated)	15	Test query returns document-grounded answers

Criterion	Marks	Description
System prompt constrains responses correctly	20	Out-of-scope questions trigger the not-found response
Memory works across conversation turns	20	Follow-up questions resolved using earlier context
Test documentation: 10 questions with analysis	15	Table shows honest assessment of correct vs. wrong answers
Chatbot embedded in Appsmith with working UI	15	Iframe loads and chat is interactive from within Appsmith

LAB 9 Integrated Multi-Tool Application: Campus Event Portal

Tools: Appsmith · NocoDB · n8n · Formbricks

Duration	3 hours
Tools	All four: Appsmith, NocoDB, n8n, Formbricks
Prerequisite	Labs 1–4 completed. All tools tested and working.
Difficulty	Medium-High

Objective

Build a complete Campus Event Portal that integrates four LCNC tools into one coherent system: NocoDB for data, Formbricks for public registration and feedback forms, n8n for event-driven automation (confirmation, reminders, feedback triggers), and Appsmith for the admin dashboard. No data should be entered twice — all four tools must communicate with each other.

System Architecture

Before building, understand the data flow:

- Formbricks Registration Form → (webhook) → n8n → NocoDB Registrations table
- n8n polls NocoDB → sends confirmation email to participant → sends admin notification
- n8n scheduled job → 1 hour before event → sends reminder email to all confirmed registrants
- n8n detects event ended → triggers Formbricks feedback form link via email
- Appsmith Admin Dashboard → reads NocoDB (Events, Registrations) → admin approves/rejects → updates NocoDB
- Capacity check: when Registrations count = Venue capacity → n8n sets a NocoDB flag → Formbricks form checks flag via API

Part A: NocoDB — Data Schema

Use the Events, Venues, Participants, and Registrations tables from Lab 1. Add these additional fields:

Step 1: In Events table, add:

- Registration Closed — Checkbox (set to true when capacity reached)
- Feedback Form URL — URL (store the Formbricks feedback form link here)

Step 2: In Registrations table, add:

- Confirmation Sent — Checkbox (default: false)
- Reminder Sent — Checkbox (default: false)
- Feedback Sent — Checkbox (default: false)

Part B: Formbricks — Registration Form

Step 1: Create a new survey in Formbricks: 'Event Registration Form'.

Step 2: Add questions: Full Name (text), USN (text), Email (email), Branch (select), Event Name (text, pre-fill via URL param if possible), Dietary Preference (select: Veg/Non-Veg/Vegan).

Step 3: Add a custom ending: 'You will receive a confirmation email shortly.' with redirect to college website.

Step 4: Configure webhook: POST to n8n webhook URL (create the webhook node in n8n first).

Step 5: Create a second survey: 'Post-Event Feedback Form'.

Step 6: Add questions: Overall Rating (1-5 stars), Best Part (long text), Improvement Suggestion (long text), Would Attend Again (yes/no).

Step 7: Save and copy the public link — you will store this in the Events table.

Part C: n8n — Build All 4 Automation Workflows

Workflow 1: Registration Confirmation

Step 1: Trigger: Webhook (receives Formbricks registration submission).

Step 2: Node 2: POST to NocoDB — create a new Registrations record linking to the Event and a new/existing Participant.

- Participant: first check if USN exists (GET /Participants?where=(USN,eq,{{usn}})). If exists, use that ID. If not, POST new participant.

Step 3: Node 3: Check if Registrations count for this Event $\geq$ Venue capacity.

- GET /Registrations with filter on Event ID and count results.
- If count $\geq$ capacity: PATCH Event record to set Registration_Closed = true.

Step 4: Node 4: Send confirmation email (Mailpit) to participant with event details.

Step 5: Node 5: PATCH Registrations record — set Confirmation_Sent = true.

Workflow 2: Reminder (1 Hour Before Event)

Step 1: Trigger: Schedule — CRON: 0 * * * * (every hour).

Step 2: GET all Events where Event_Date = today and Start_Time is within the next 60 minutes.

Step 3: For each event, GET all Registrations where Event = this event ID and Reminder_Sent = false and Status = Confirmed.

Step 4: For each registration, send reminder email with event location, time, and what to bring.

Step 5: PATCH each Registrations record: Reminder_Sent = true.

Workflow 3: Post-Event Feedback Trigger

Step 1: Trigger: Schedule — CRON: 30 * * * * (every 30 minutes).

Step 2: GET Events where End_Time is in the past 30 minutes and Feedback_Sent is not true (check a new Events field 'Feedback_Triggered').

Step 3: GET all Confirmed Registrations for that event where Feedback_Sent = false.

Step 4: Send email to each participant with the Feedback Form URL (from Events table).

Step 5: PATCH Registrations: Feedback_Sent = true. PATCH Event: Feedback_Triggered = true.

Workflow 4: Admin Notification on New Registration

Step 1: Add a branch in Workflow 1 (after creating the registration) that sends a summary email to admin.

Step 2: Email body: 'New registration: [Name] ([USN]) for [Event Name]. Current count: X / Y capacity.'

Part D: Appsmith — Admin Dashboard

Step 1: Create a new Appsmith app: 'Event Portal Admin'.

Step 2: Page 1: Events Overview

- Table showing all events with columns: Name, Date, Venue, Registered Count, Capacity, Registration Closed status.
- 'Registered Count' is a computed column: fetch registration count per event via a separate API call.
- 'Toggle Registration' button — PATCH Event to flip Registration_Closed boolean.
- Row click navigates to Page 2 (Registration Details) passing the event ID.

Step 3: Page 2: Registration Details

- Header showing selected event name, date, venue.
- Table of all registrations for this event: Name, USN, Branch, Status, Confirmation Sent, Reminder Sent.
- Status column editable inline — click status cell → dropdown to change Pending/Confirmed/Cancelled.
- 'Approve All Pending' button — runs a loop query updating all Pending to Confirmed.
- 'Export CSV' button — use Appsmith's downloadFile function to export the registrations table as CSV.

Part E: Capacity Check Integration

This is the most complex integration — the registration form must become unavailable when an event is full.

Step 1: In Formbricks, on the Registration Form, add a 'Welcome Card' with a hidden field that reads a URL parameter: ?event_id=

Step 2: In n8n Workflow 1, after setting Registration_Closed = true, make an additional HTTP request to the NocoDB Events table to confirm the flag is set.

Step 3: The Formbricks form itself cannot read NocoDB directly. Work around: In Appsmith, add a 'Register' button per event that:

- Checks if Registration_Closed === true → shows an error toast: 'This event is full. Registration is closed.'
- If not full → opens the Formbricks form URL in a new tab with event_id as a URL parameter.

End-to-End Test

Step 1: Use Appsmith to open the Formbricks registration form for an event.

Step 2: Submit a registration. Verify: NocoDB record created, confirmation email in Mailpit, admin notification email in Mailpit.

Step 3: Fill registrations until the event reaches capacity. Verify: Registration_Closed flag set in NocoDB, Appsmith shows error toast on further registration attempts.

Step 4: Manually trigger the reminder workflow. Verify reminder emails appear in Mailpit.

OUTCOME: Students have built a genuine multi-tool system integration where four platforms collaborate through webhooks, REST APIs, and shared data. This mirrors how real enterprise software stacks operate — with NocoDB as the source of truth, n8n as the event bus, Formbricks as the public interface, and Appsmith as the operations dashboard.

Evaluation Rubric

Criterion	Marks	Description
Registration webhook triggers n8n and creates NocoDB record	20	Record visible in NocoDB within 5 seconds of form submission
Confirmation email sent with correct event details	15	Email visible in Mailpit with participant name and event info
Reminder workflow correctly targets upcoming events	20	Reminder emails sent only to unreminded registrants of today's events
Feedback trigger fires after event end time	15	Feedback emails sent with correct form link after event completion
Appsmith dashboard shows live data with inline editing	15	Status updates from Appsmith reflected in NocoDB immediately
Capacity check prevents over-registration	15	Appsmith blocks registration and shows error when event is full

LAB 10 Product Sprint: Design, Build & Demo a Complete LCNC Product

Tools: All Tools

Duration	3 hours (structured sprint with milestones)
Tools	All — Appsmith, NocoDB, n8n, Formbricks, Metabase, GrapesJS (student choice)
Prerequisite	Labs 1–9 completed
Difficulty	Medium-High
Format	Group work (3 students per group)

Objective

Ship a complete, working LCNC product in 3 hours under timed milestone constraints. Each group builds a different assigned problem from scratch, demonstrating their ability to decompose a real-world problem into data, UI, automation, and analytics layers using no-code tools.

Problem Statements (Assigned by Faculty)

Group A: Hostel Complaint Tracker

A system where students submit maintenance complaints (broken furniture, electrical issues, water problems), wardens manage resolution, and escalation happens automatically if a complaint remains unresolved after 48 hours.

- Minimum required: Submission form, complaint table with status, escalation automation, summary dashboard.

Group B: Lab Equipment Booking System

Faculty and students book lab equipment (oscilloscopes, soldering stations, 3D printers) for specific time slots. The system prevents double-booking the same equipment, sends booking confirmations, and shows utilization analytics.

- Minimum required: Booking form with conflict detection, equipment calendar view, confirmation email, utilization dashboard.

Group C: Placement Drive Portal

Students register for placement drives, upload resumes (URL), and get shortlisted by the placement coordinator. Shortlisted students receive interview schedule emails. The portal shows company-wise registration analytics.

- Minimum required: Student registration form, coordinator shortlisting UI, interview schedule email, registration analytics.

Milestone Structure

Milestones are time-boxed and reviewed by the faculty at the specified time. Groups must demo progress at each checkpoint.

Milestone	Time
M1: Design	0:00 – 0:30
M2: Core UI	0:30 – 1:30
M3: Automation	1:30 – 2:15
M4: Analytics	2:15 – 2:45
M5: Demo	2:45 – 3:00

System Diagram Requirement

Each group must submit a hand-drawn or GrapesJS-built system diagram showing all tools used, how they connect, and the data flow direction. Submit before M5 demo.

Minimum diagram elements:

- Each LCNC tool used (box with tool name)
- Arrows showing data flow direction with labels (webhook / REST API / manual / email)
- The NocoDB tables shown as a simple ER diagram
- The user roles (Student / Faculty / Admin) and which tool they interact with

Evaluation Rubric

Criterion	Marks	Description
M1: Schema design quality (tables, types, relationships)	15	Correct field types, required constraints, at least 2 linked tables
M2: UI completeness and usability	25	All 3 screens functional, forms validated, API wired correctly
M3: Automation correctness and branching	25	Both workflows active, conditional branch tested, emails delivered
M4: Analytics dashboard quality	15	4 charts correct, filter works, dashboard assembled meaningfully

Criterion	Marks	Description
M5: Demo clarity and system diagram	20	Full user journey demoed without errors; diagram shows all connections

Sprint Rules

- Each group member must own at least one tool (e.g., one person owns NocoDB + Appsmith, one owns n8n, one owns Metabase + GrapesJS).
- No pre-built templates from previous labs may be reused directly — start fresh.
- All work must be done during the 3-hour session on lab machines.
- Faculty may ask any group member to explain any part of the system — not just their own.
- Groups may consult this lab manual but not outside internet resources.

OUTCOME: Students demonstrate integrated systems thinking — the ability to decompose a real problem, select appropriate tools, design data models, build UIs, automate workflows, and present analytics — exactly the skills demanded in modern software product teams and startup environments.

Appendix: Useful Commands & Reference

Docker Commands

Command	Action
<code>docker compose up -d</code>	Start all lab services in background
<code>docker compose stop</code>	Stop all services (data preserved)
<code>docker compose start</code>	Restart stopped services
<code>docker compose ps</code>	List all containers and their status
<code>docker compose logs -f n8n</code>	Follow logs for a specific service
<code>docker compose pull</code>	Update all images to latest version
<code>docker compose down -v</code>	Stop and delete all data (CAUTION: destructive)
<code>docker exec -it lcnc_ollama ollama pull llama3</code>	Download Llama 3 model
<code>docker exec -it lcnc_ollama ollama list</code>	List downloaded Ollama models

NocoDB API Quick Reference

Operation	Endpoint & Method
List records	GET /api/v1/db/data/noco/{projectId}/{tableName}
Get one record	GET /api/v1/db/data/noco/{projectId}/{tableName}/{rowId}
Create record	POST /api/v1/db/data/noco/{projectId}/{tableName}
Update record	PATCH /api/v1/db/data/noco/{projectId}/{tableName}/{rowId}
Delete record	DELETE /api/v1/db/data/noco/{projectId}/{tableName}/{rowId}
Filter records	?where=(field,eq,value)&limit=50&offset=0
Sort records	?sort=-fieldName (descending) or sort=fieldName (ascending)
Auth header	xc-token: {your API token from NocoDB Settings}

Service Access URLs

Service	URL
NocoDB	http://localhost:8080

Appsmith	http://localhost:8081
n8n	http://localhost:5678 (admin / admin123)
Formbricks	http://localhost:3000
Node-RED	http://localhost:1880
Node-RED Dashboard	http://localhost:1880/ui
Metabase	http://localhost:3001
Flowise	http://localhost:3002 (admin / admin123)
Mailpit (Email Inbox)	http://localhost:8025
Ollama API	http://localhost:11434

Troubleshooting

Problem	Solution
Container not starting	Run: docker compose logs {service-name} to see the error
NocoDB shows blank page	Wait 60 seconds after docker compose up — NocoDB takes time to initialize
Appsmith shows error	Clear browser cache and hard reload (Ctrl+Shift+R)
n8n webhook not receiving	Ensure n8n is not in test mode — activate the workflow
Ollama model not responding	Run: docker exec -it lcnc_ollama ollama list — model must be downloaded
Metabase cannot connect to DB	Confirm host is 'postgres' not 'localhost' in Metabase DB settings
Port already in use	Run: sudo lsof -i :{port} then kill the conflicting process
Docker volume full	Run: docker system prune (removes unused images, not volumes)